

Card Transaction Report

Table of contents

Executive Summary.....	3
Description of the Data.....	4
Data Description.....	4
Field Summary.....	4
Field-by-Field Distribution Plots.....	5
Data Cleaning.....	7
Outlier.....	7
Exclusions.....	7
Imputation (Missing Value Handling).....	7
Variable Creation.....	9
Feature Selection.....	10
Experiment Setup.....	10
Result of Each Run.....	10
Preliminary Model Explores.....	19
Final Model Performance.....	23
Financial Curves and Recommended Cutoff.....	27
Summary.....	28
Appendix-Full DQR.....	29

Executive Summary

In this project, I developed and tested several machine learning models to help identify fraudulent credit card transactions more effectively. The goal was to rank transactions by likelihood of fraud, enabling the business to investigate only the top few percent of risky cases while recovering the majority of fraud losses. After evaluating multiple model types and tuning their parameters, I selected an XGBoost model as my final solution due to its strong and consistent performance.

Using this model, I was able to identify approximately **78% of fraud cases** by reviewing just the **top 3% of all transactions**. When applied to out-of-time (OOT) data, this translates to an **estimated \$59.4 million in potential fraud savings**, while only investigating a small fraction of daily volume. This model can provide significant financial benefit while keeping investigation resources focused and efficient.

Description of the Data

Data Description

This dataset contains 98,393 credit card transaction records collected from a U.S. government organization, covering a time period from February to October 2010. It includes 10 fields such as transaction date, amount, merchant details, and a binary Fraud label. The data is moderately messy and reflects real-world fraud detection challenges, including missing values, skewed distributions, and ID-like fields that require careful handling during modeling.

Field Summary

Field Name	Field Type	Records Have Value	%Populated	#Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98,393	100%	0	0.01	3,102,046	424.29092	9,922.44	3.62

Table 1. Numeric Field Summary Table

Field Name	Field Type	Records Have Value	%Populated	#Zeros	Unique Values	Most Common
Date	categorical	98,393	100%	0	365	2/28/10
Merchnum	categorical	94,970	96.52%	0	13,091	930090121224
Merch description	categorical	98,393	100%	0	13,126	GSA-Fss-ADV
Merch state	categorical	97,181	98.77%	0	227	TN
Transtype	categorical	98,393	100%	0	4	P
Recnum	categorical	98,393	100%	0	98,393	1
Fraud	categorical	98,393	100%	95901	2	0
Merch zip	categorical	93,664	95.19%	0	4,567	38118
Cardnum	categorical	98,393	100%	0	1,645	5142148452

Table 2. Categorical Field Summary Table

Field-by-Field Distribution Plots

Transtype

The Transtype field indicates the type of transaction, such as purchase, adjustment, or credit. It is a categorical variable with a strong class imbalance, where one value significantly dominates the distribution. To address this imbalance and enable meaningful comparisons across all categories, a logarithmic y-axis was used in the plot, which helps highlight the presence of low-frequency values that would otherwise be visually obscured.

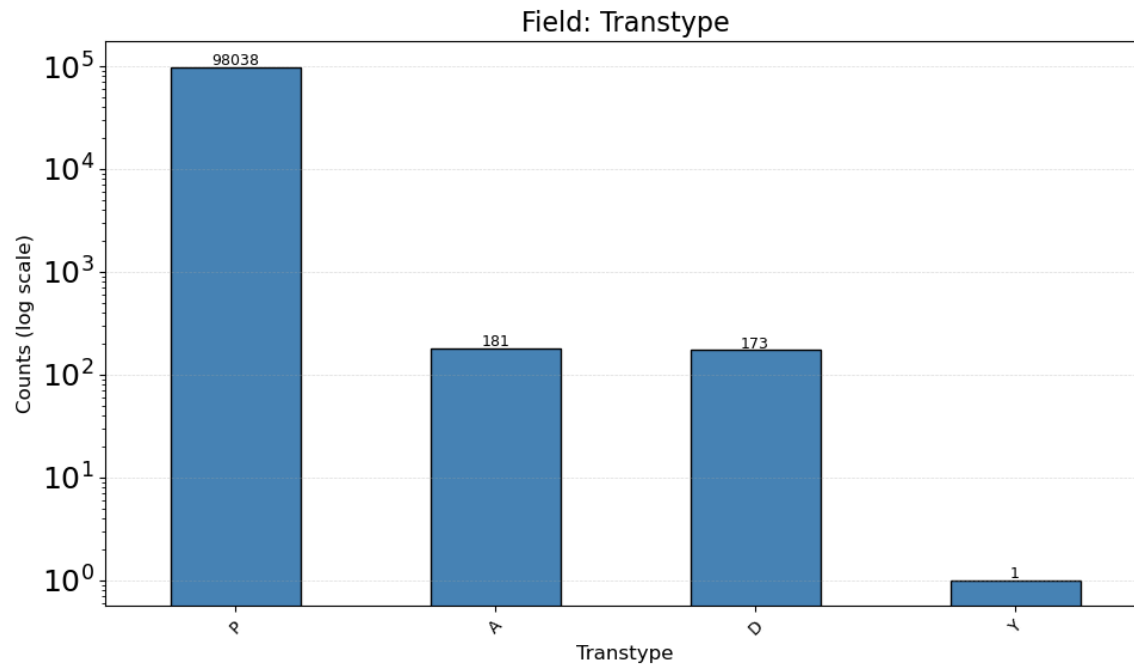


Figure 1. Transtype Distribution

Amount

The Amount field represents the dollar value of each credit card transaction and is a numeric field with a highly right-skewed distribution—most transactions are under \$1,000, while a small number reach much higher values. To make the distribution more interpretable, the data was clipped at the 99.5th percentile to exclude extreme outliers and focus on typical transaction values. A logarithmic scale was applied to the y-axis (frequency) to highlight differences in density across both common and rare transaction amounts, while the x-axis remains on the original dollar scale. This approach effectively reveals the true shape of the transaction distribution without distortion from a few exceptionally large values.

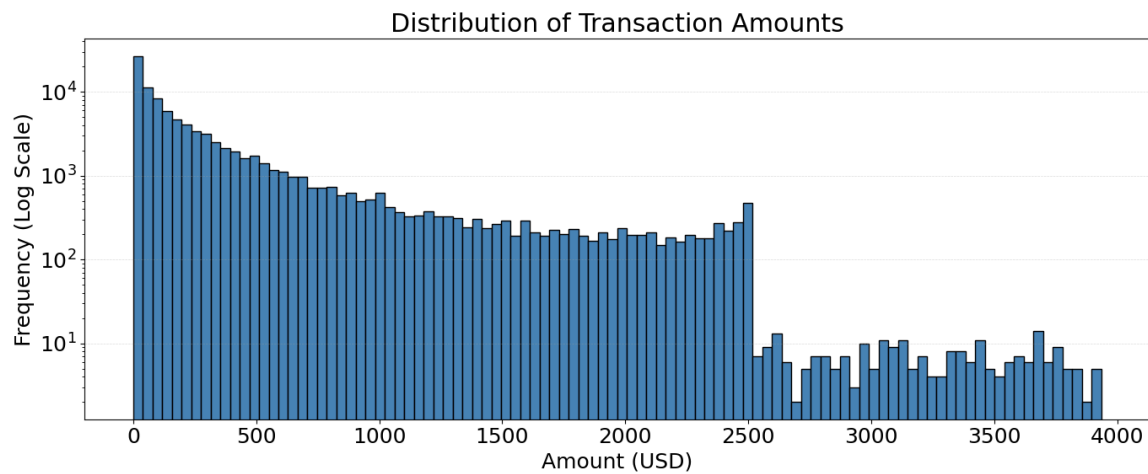


Figure 2. Transaction Amount Distribution

Fraud

The Fraud field is a binary categorical variable indicating whether a transaction was fraudulent (1) or not (0). Out of the 98,393 total records, only 2,492 (approximately 2.5%) are labeled as fraudulent, while the remaining 95,901 are legitimate. This represents a significant class imbalance, which is typical in fraud detection datasets and presents modeling challenges. The bar chart clearly shows the overwhelming dominance of non-fraudulent transactions. Any modeling efforts will require special consideration of this imbalance (e.g., resampling, using specialized evaluation metrics).

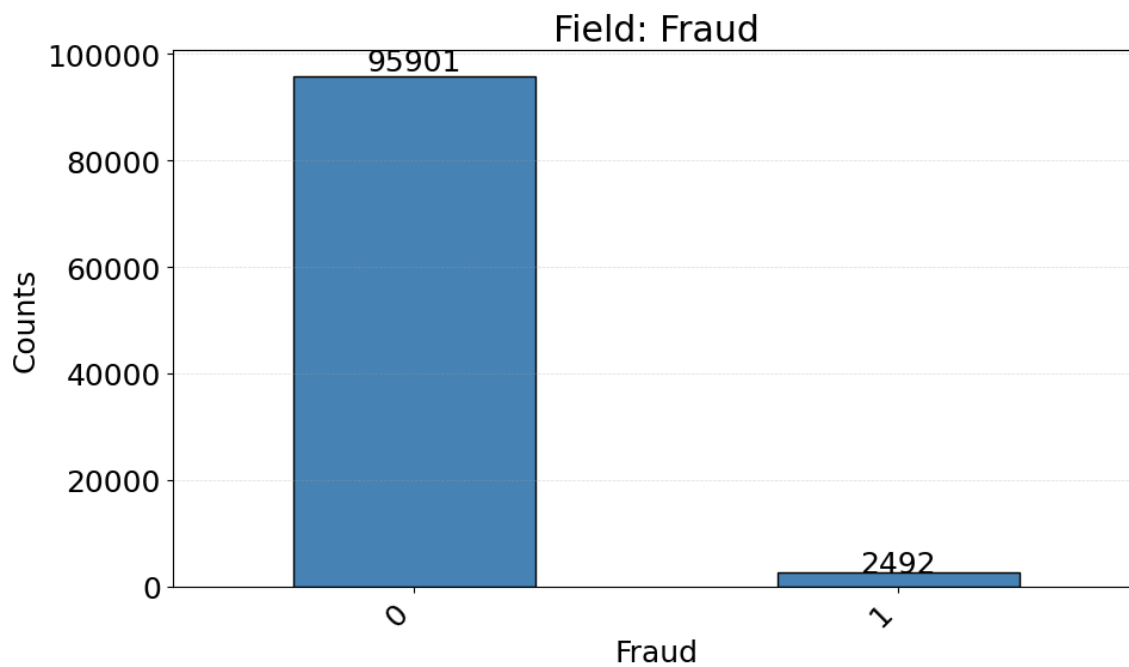


Figure 3. Fraud Distribution

Data Cleaning

Outlier

I identified a single transaction with an amount exceeding \$3,000,000, which was far beyond the rest of the dataset and considered an extreme outlier. This record was removed from the dataset using a filter that keeps only transactions where the amount is less than or equal to \$3,000,000. The purpose of this step is to prevent the extremely large value from distorting the scale of numerical features or misleading the model during training. Other outliers were retained, as they may represent valid anomalies indicative of fraud.

Exclusions

Only transactions of type 'P' (likely “purchase”) were retained for modeling. Other types ('A', 'D', and 'Y') were excluded, as their meanings were unclear and they represented non-standard or infrequent transaction types. This ensures model focus on standard transaction behavior.

Imputation (Missing Value Handling)

- Merchnum

I identified that some Merchnum values were either missing or coded as '0', which is not a valid merchant ID. We replaced '0' with missing and then imputed missing values using the most frequent Merchnum associated with the same Merch description. This ensures that each transaction has a valid and meaningful merchant identifier.

- Merch Zip

There are 4,729 records where Merch zip is missing. First, I used the Merch state to assign the most common zip code within each state, which filled 2,902 records. Then, for records where Merch state is also missing, we assigned the value "unknown", covering 918 more rows. For the last 909 records, I created placeholder zip codes based on the combination of Merch description and city. This way, all records now have a value in the Merch zip field.

- Merch State

I found 1,212 missing values in the Merch state field. To fill them, we used the most common state linked to each Merch zip, which completed 897 rows. The remaining 315 records had missing zip codes as well, so we filled those with "unknown" to ensure no missing values remain.

- Transtype

There were 217 missing values in Transtype. We filled 173 of them using the most common value based on the Merch description. The remaining 44 were filled with "P" (purchase), which is the most frequent type overall.

- Age

There are 190 missing values in the Age field. I used the average age within the same gender to fill 160 of them. For the remaining 30 records where gender is also missing, I used the overall median age from the dataset. This gives us a complete Age field.

- Gender

I found 98 missing values in Gender. Of these, 73 were filled using the most common gender for the same demo_id. For the rest (25 rows), we set the value to "unknown" so that no data is lost, while clearly marking the missing values.

Variable Creation

Description	Variables Created
Day Since: Days since a transaction with the same Cardnum, Merchnum, Zip, State, or combo fields.	23
Velocity: Number of transactions with the same entity over {0,1,3,7,14,30,60} days	126
Relative Velocity: Today's count ÷ rolling average for selected entities	184
Counts by Entity: Unique counts over {0,1,3,7,14,30,60} days for ~23 entities × multiple groupings	3,542
Maximum Indicator: Highest transaction count seen over time per entity	92
Target Encoding: Smoothed fraud rate for Merchnum, Zip, State	3
Benford's Law Features (U)*: Log-digit-based fraud detection metrics(not suitable for real time scoring algorithm)	2
Date-Based Features: Is_Weekend, Dow_TE, Dow_Risk	3
Entity Variability Stats: Rolling avg, max, median of Amount over time windows {0,1,3,7,14,30} × 23 entities × 3 metrics	3,276
Custom Features: Binned amount (amount_cat), merchant mismatch flag, high-risk merchant flag, etc.	9
isForeign: Flag that indicates whether the transaction occurred in a different state than the cardholder's home state	1
Time Pattern Features: These features are extracted to capture patterns in transaction timing. They include (1) Hour of transaction: derived from the timestamp to detect activity trends across the day; (2) Same hour transaction count: number of transactions a card made in the same hour to identify bursty behavior; (3) Short interval frequency: counts of transactions by card over recent intervals; (4) Week to week ratio: comparing this week's transaction count to last week for the same card or merchant	4
Weekday-Weekend Shift: Indicates if a card's current transaction behavior has shifted compared to past (e.g., card usually active on weekdays but now active on weekend).	1
High-Frequency Bursts: Binary flag that equals 1 if a card has ≥ 3 transactions within a 5-minute window — detects rapid fraud attempts.	1
Card Merchant Familiarity: Count of previous transactions between the same Cardnum and Merchnum, used to identify first-time or unusual pairings.	1

Table 3. Variable Creation

Feature Selection

To improve model performance and reduce noise from irrelevant features, I explored multiple combinations of feature selection techniques—first using a filter method to select top-ranked variables based on univariate scores, and then applying a wrapper method to select the best subset of ~20 variables. This process allowed me to identify a diverse and high-performing set of features for modeling.

Experiment Setup

Filter Top-k	Wrapper Model	Selection Type	Final Variables	Runtime
500	LGBM Classifier	Forward	20	22min 28sec
500	CatBoost Classifier	Forward	20	36min 32sec
500	RandomForest Classifier	Forward	20	20min 26sec
300	LGBM Classifier	Forward	20	22min 25sec
300	CatBoost Classifier	Forward	20	29min 51sec
200	LGBM Classifier	Backward	20	5hr 04min 01sec

Table 4. Wrapper Feature Selection Configuration and Runtime Summary

Result of Each Run

1. 500/LGBM/Forward

This model shows a rapid increase in FDR@3% in the first 5–6 variables, peaking at around **0.70** by the 9th feature. Performance then flattens with very minimal improvement beyond 10 variables, indicating a clear saturation point. The curve is smooth and stable — making this run a strong and reliable baseline.

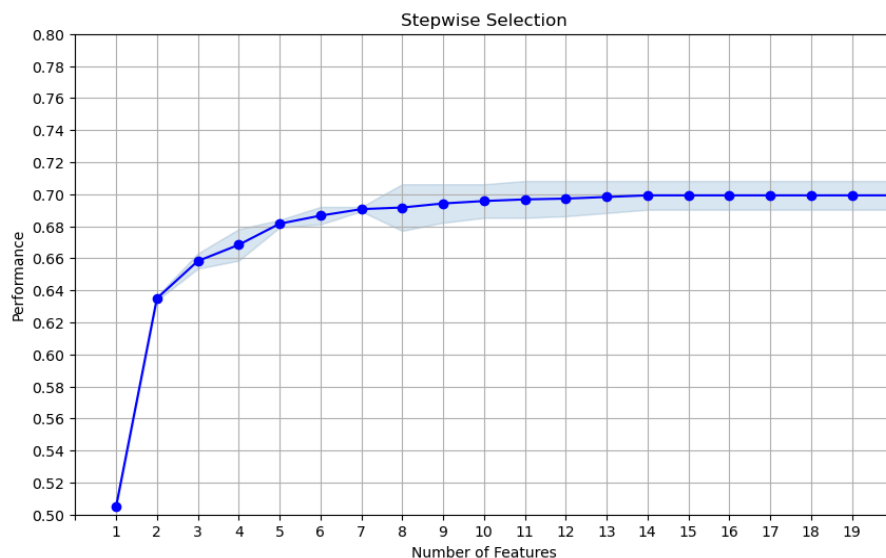


Figure 4. Forward Selection Performance Curve — 500 Top-K Filter with LGBM Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchnum_desc_total_14	0.26558329883066800
3	card_state_total_amount_1_by_60	0.3488908916449490
4	card_state_max_14	0.2293557922186700
5	Card_dow_vdratio_0by30	0.5076936391446800
6	Cardnum_total_amount_1_by_60	0.4574137668325220
7	card_zip_count_0_by_60	0.2642663413543190
8	merch_zip_total_1	0.24215877603657700
9	Card_dow_unique_count_for_merch_state_14	0.41787101399187300
10	Cardnum_actual/toal_1	0.4755878953142160
11	Cardnum_variability_med_0	0.26918950785314400
12	Merchnum_desc_total_3	0.24915661971242000
13	card_state_total_1	0.2954647564386410
14	Cardnum_vdratio_0by14	0.4017270867195840
15	Card_dow_vdratio_0by7	0.490635686388675
16	Cardnum_unique_count_for_card_zip_7	0.47442357530392700
17	Cardnum_unique_count_for_Merchnum_7	0.4680367503273130
18	Cardnum_day_since	0.45108180188490700
19	Card_dow_day_since	0.45108180188490700
20	Cardnum_unique_count_for_card_zip_14	0.43560436789481400

Table 4. Top 20 Selected Variables from Forward Selection with LGBM (500 Filter Top-K)

2. 500/CatBoost/Forward

Performance improved quickly in the first 6 to 7 variables, reaching a peak around **0.715**. After that, the curve becomes flatter, with only small improvements. The performance stayed strong overall, but there were small ups and downs after the 10th variable. This shows that most of the useful information was captured early, and later variables may be less helpful or a bit noisy. In general, this run gave good results with strong early performance and stable feature selection.

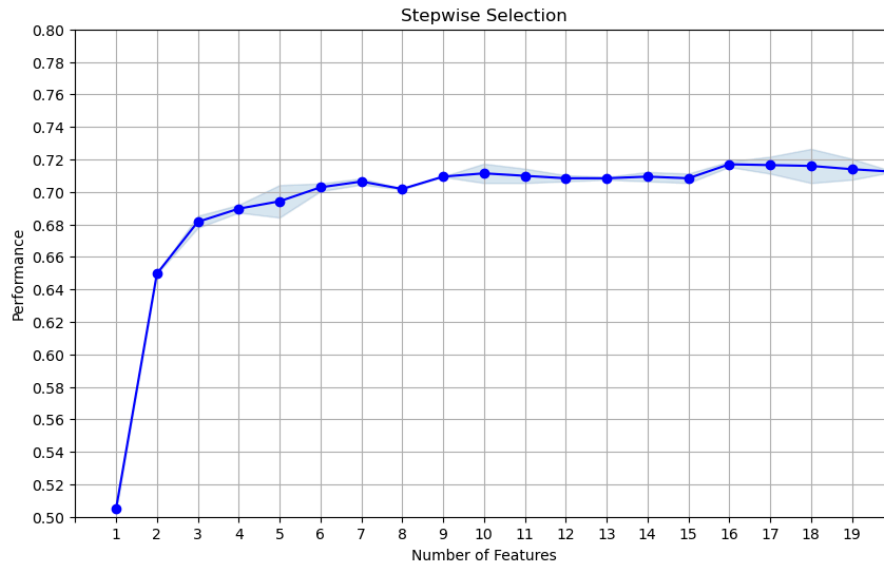


Figure 5. Forward Selection Performance Curve — 500 Top-K Filter with CatBoost Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchdesc_State_total_3	0.2599209696267790
3	Card_Merchdesc_Zip_count_1_by_60_sq	0.22060669147901900
4	Cardnum_max_0	0.49231631409536500
5	Card_Merchnum_State_total_14	0.265345555143389
6	Card_dow_count_0_by_60	0.37386348222117500
7	Card_Merchdesc_State_total_7	0.2631521043246540
8	card_zip_max_30	0.2154629001055200
9	High_Amount_For_Merchant	0.2515712466037180
10	Cardnum_max_1	0.5048046652882500
11	Card_Merchdesc_State_avg_7	0.21459924270478500
12	Card_Merchdesc_Zip_total_30	0.2533196653151760
13	card_zip_total_7	0.263851061261844
14	Card_dow_count_0_by_60_sq	0.37386348222117500
15	Card_dow_unique_count_for_state_des_30	0.33894308121520800
16	Card_dow_count_0_by_14_sq	0.25581156128069200
17	card_merch_total_amount_1_by_60	0.3055671810441150
18	Cardnum_count_0	0.5290724258187430
19	merch_state_total_1	0.243172479639732
20	Cardnum_total_3	0.569584754753369

Table 5. Top 20 Selected Variables from Forward Selection with CatBoost (500 Filter Top-K)

3. 500/RandomForest/Forward

Performance climbs quickly in the first 6–7 variables, stabilizing around **0.69**. However, the curve shows slight instability and noise after 10 variables. While performance is solid, the increased variance implies potential overfitting or suboptimal ordering during forward selection.

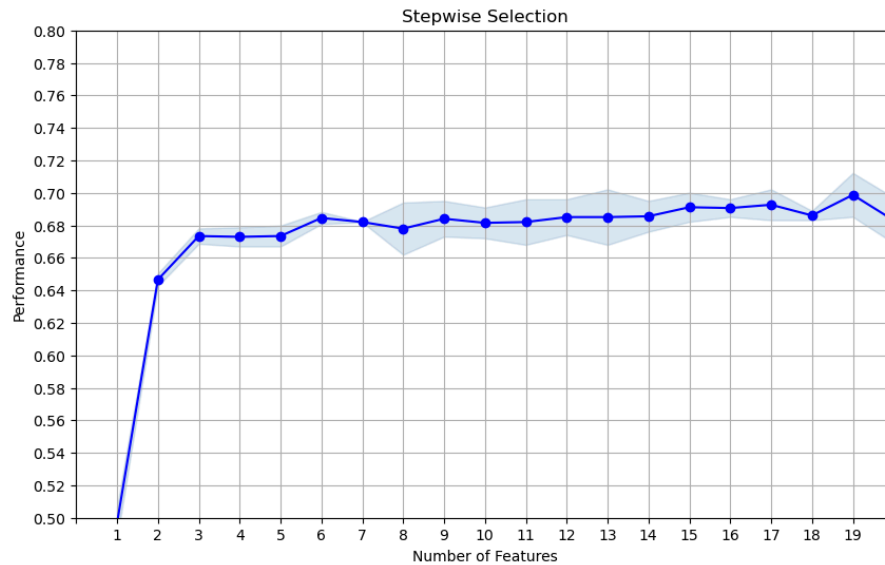


Figure 6. Forward Selection Performance Curve — 500 Top-K Filter with RandomForest Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchdesc_State_count_1_by_60	0.22145174753489400
3	Card_Merchdesc_State_total_14	0.2673557168249350
4	Cardnum_total_amount_0_by_30	0.33047395732524600
5	Merchnum_total_0	0.23705368889187400
6	card_zip_total_amount_1_by_30	0.281011201038026
7	Card_dow_count_7	0.4798436118540490
8	card_state_total_amount_1_by_60	0.3488908916449490
9	Card_dow_unique_count_for_merch_zip_1	0.49073745525872900
10	Card_Merchnum_desc_total_30	0.2543774394070380
11	card_state_total_3	0.291571341637837
12	Merchdesc_Zip_total_3	0.25211054622126900
13	Card_Merchdesc_Zip_max_1	0.22932542564989900
14	Card_Merchdesc_count_1_by_60	0.2217337264366860
15	card_zip_count_1_by_30	0.27378390698567300
16	Merchnum_total_1	0.2431599668140860
17	Card_Merchdesc_State_count_1_by_60_sq	0.22145174753489400

18	Cardnum_total_60	0.2876972314343720
19	Card_Merchdesc_avg_3	0.2185832566988760
20	Card_Merchdesc_Zip_count_1_by_60	0.22060669147901900

Table 6. Top 20 Selected Variables from Forward Selection with RandomForest (500 Filter Top-K)

4. 300/LGBM/Forward

Performance improved steadily in the first few variables, reaching about **0.70** by the 9th feature. After that, the curve flattened and remained stable through the remaining selections. There were no major ups and downs, and the model performance stayed consistent, with a clear saturation point starting around the 10th variable. This shows that the top features were selected early, and later variables added only small improvements. Overall, this run gave solid results with a clean performance curve and efficient variable selection.

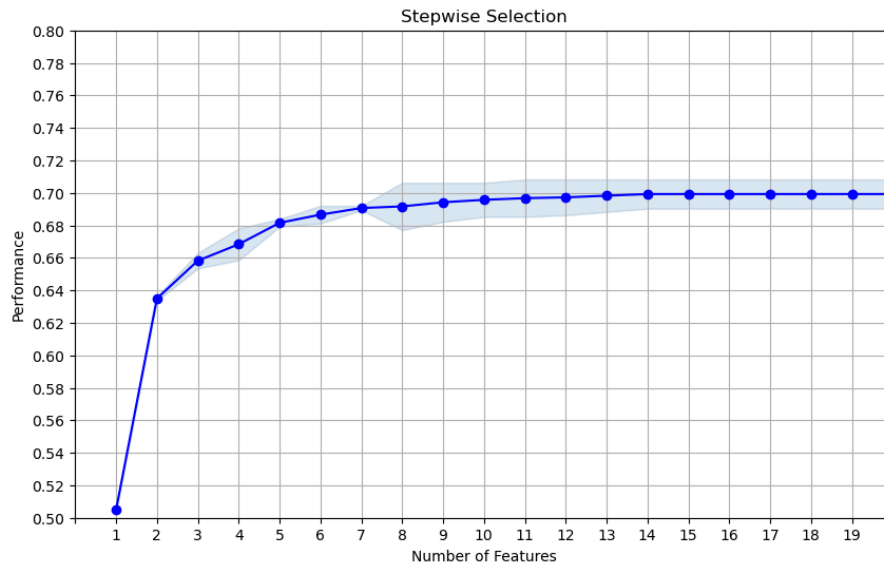


Figure 7. Forward Selection Performance Curve — 300 Top-K Filter with LGBM Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchnum_desc_total_14	0.26558329883066800
3	card_state_total_amount_1_by_60	0.3488908916449490
4	Card_dow_count_0_by_14_sq	0.25581156128069200
5	High_Amount_For_Merchant	0.2515712466037180
6	Cardnum_count_0_by_30_sq	0.30860426874255000
7	Cardnum_unique_count_for_card_state_7	0.47619034193592700
8	Card_dow_unique_count_for_merch_state_30	0.37722300801359400
9	Card_dow_unique_count_for_merch_zip_30	0.37706034128019200
10	Card_dow_unique_count_for_merch_zip_60	0.31927203864974300
11	Cardnum_count_1_by_7_sq	0.31810073679952300
12	Cardnum_count_1_by_7	0.31810073679952300

13	Cardnum_count_0_by_30	0.30860426874255000
14	card_state_count_1_by_30_sq	0.30483642569076300
15	card_state_count_1_by_30	0.30483642569076300
16	Cardnum_total_amount_1_by_7	0.256441257352077
17	Card_dow_count_0_by_14	0.25581156128069200
18	Cardnum_unique_count_for_card_zip_7	0.47442357530392700
19	card_state_total_amount_1_by_30	0.29984290860157000
20	Card_dow_unique_count_for_merch_state_14	0.41787101399187300

Table 7. Top 20 Selected Variables from Forward Selection with LGBM (300 Filter Top-K)

5. 300/CatBoost/Forward

This model delivered the **best overall performance**, peaking at around **0.715** by the 11th variable. The curve is smooth and consistently improving up to saturation. The result is highly stable and reflects a strong, diverse variable set. Runtime was acceptable (≈ 27 minutes), and the selected variables were well balanced across entity types and time windows.

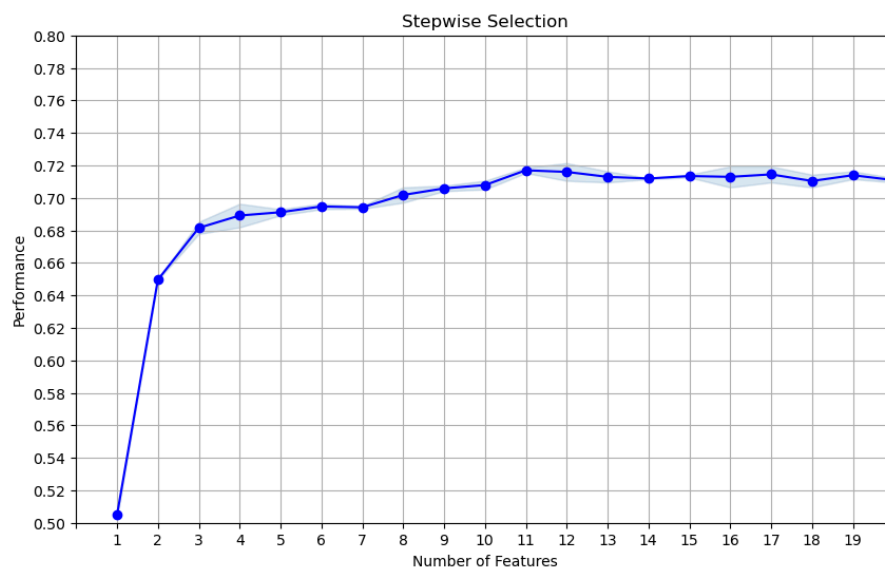


Figure 8. Forward Selection Performance Curve — 300 Top-K Filter with CatBoost Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchdesc_State_total_3	0.2599209696267790
3	Card_Merchnum_Zip_total_amount_1_by_60	0.3058974499494890
4	Cardnum_max_0	0.49231631409536500
5	card_merch_total_7	0.2598647537776860
6	Cardnum_total_amount_0_by_60	0.37029630054350000
7	card_zip_total_amount_0_by_60	0.2655482819256830
8	card_state_max_7	0.25395582370664100

9	Card_dow_vdratio_0by14	0.5026571349557350
10	Card_dow_total_30	0.42680301106102100
11	High_Amount_For_Merchant	0.2515712466037180
12	Cardnum_variability_med_0	0.26918950785314400
13	Card_dow_avg_60	0.25100649051353600
14	card_state_total_14	0.2642250521974900
15	Card_dow_actual/toal_14	0.3370331741934220
16	Card_dow_unique_count_for_Card_Merchdesc_1	0.48688598218237000
17	Card_dow_count_0_by_60	0.37386348222117500
18	Merchnum_desc_State_total_3	0.24915661971242000
19	Cardnum_unique_count_for_card_state_60	0.32730762322519800
20	Cardnum_variability_max_3	0.4159074760490260

Table 8. Top 20 Selected Variables from Forward Selection with CatBoost (300 Filter Top-K)

6. 200/LGBM/Backward

Performance improves steadily over the first several variables, reaching a peak around 0.675. However, unlike earlier runs, the curve fluctuates slightly throughout, and the performance shows some inconsistency in the second half of the selection process. The improvement beyond 10 features is relatively small, and the final performance is slightly lower than the top forward-selection runs. Additionally, this run had the longest runtime (over 5 hours), which may not be practical compared to the much faster forward runs that delivered similar or better performance. This suggests that backward selection may not be necessary in this case, especially when forward methods already produce strong, stable results.

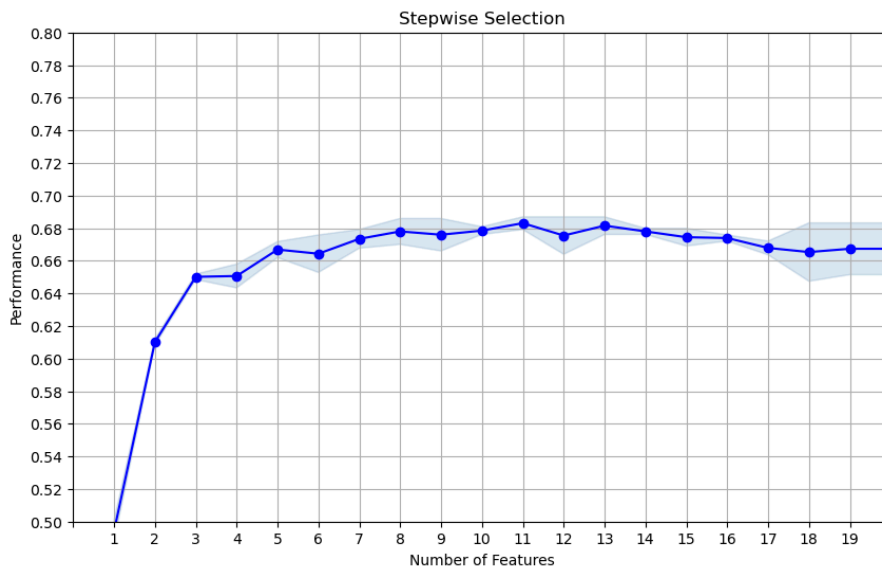


Figure 8. Backward Selection Performance Curve — 300 Top-K Filter with LGBM Classifier

wrapper order	variable	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	card_zip_total_amount_1_by_60	0.34588508284282500
3	Card_dow_vdratio_0by60	0.5084477665556560
4	Cardnum_total_7	0.5159873870717490
5	Cardnum_total_30	0.36910954614555900
6	Cardnum_unique_count_for_card_zip_60	0.3014462545628260
7	card_merch_vdratio_1by30	0.2733331094751870
8	card_merch_vdratio_0by30	0.2733331094751870
9	card_merch_day_since	0.27337064795212600
10	card_merch_vdratio_0by7	0.2733706479521260
11	card_merch_vdratio_1by14	0.2733706479521260
12	card_merch_vdratio_1by7	0.2733706479521260
13	card_merch_vdratio_0by14	0.2733706479521260
14	card_zip_count_1_by_30_sq	0.27378390698567300
15	Card_dow_unique_count_for_state_des_60	0.2747260118837000
16	Card_dow_unique_count_for_Card_Merchdesc_60	0.274776063186285
17	card_zip_vdratio_0by60	0.2762725401131100
18	card_zip_vdratio_1by60	0.2762725401131100
19	card_zip_vdratio_0by30	0.27643520684651100
20	card_zip_vdratio_1by30	0.27643520684651100

Table 9. Top 20 Selected Variables from Backward Selection with LGBM (300 Filter Top-K)

Final Variable Set

After comparing performance and runtime across all runs, I selected the final variable set from **Run 5: CatBoost + Forward Selection + Filter Top 300**. This run achieved the highest FDR@3% (~0.715) and had a smooth performance curve that leveled off after about 11 variables. The selected variables cover a wide range of patterns, including (1) card-location features like `card_state_*` and `card_zip_*`, (2) time-based variables such as `Merchnum_day_since`, (3) frequency and amount-based features over different time windows (like `count_7`, `total_amount_0_by_60`, and `actual/max_1`), and (4) more complex combinations such as `Card_Merchdesc_Zip_max_1` and `state_des_total_1`. These variables include velocity, time lags, aggregates, and relative ratios, and they reflect different time windows and behaviors. Overall, this variable set is diverse, easy to understand, and performed well under CatBoost, so I selected it as the final set for modeling.

Wrapper order	variable name	filter score
1	Cardnum_unique_count_for_card_state_1	0.5186059951957090
2	Card_Merchdesc_total_7	0.2631145658477150
3	card_zip_total_amount_0_by_60	0.2655482819256830
4	Cardnum_max_1	0.5048046652882500
5	state_des_total_3	0.25387149676519700
6	card_state_total_amount_1_by_30	0.29984290860157000
7	Card_dow_total_amount_0_by_60	0.32686028495663800
8	Cardnum_avg_7	0.2644578160969420
9	Card_Merchdesc_State_total_7	0.263152104324654
10	Card_Merchnum_Zip_total_amount_1_by_60	0.305897449949489
11	Card_dow_vdratio_0by30	0.50769363914468
12	Cardnum_avg_7	0.264457816096942
13	card_merch_total_14	0.265345555143389
14	Card_dow_total_7	0.508754460663953
15	Cardnum_max_3	0.466271878042082
16	Cardnum_avg_3	0.303771517704381
17	Card_Merchdesc_Zip_total_7	0.262708339338962
18	Cardnum_total_1	0.591113714342011
19	Card_Merchnum_Zip_total_amount_1_by_30	0.249383016325911
20	Cardnum_unique_count_for_card_zip_1	0.517208448786588

Table 10. Final Variables Set

Preliminary Model Explores

Decision Tree

A simple and interpretable model that splits the data based on feature values to create decision rules. It can easily overfit when the tree grows too deep, especially with limited regularization. In our experiments, deeper trees showed higher training FDR but weaker OOT performance.

Model		Parameters				Average FDR at 3%		
Decision Tree	Iteration	max_depth	min_samples_split	Min_samples_leaf	criterion	Tra	Tst	OOT
	1	5	20	10	gini	0.671	0.664	0.538
	2	6	20	10	entropy	0.706	0.671	0.518
	3	6	40	15	gini	0.690	0.660	0.552
	4	8	40	25	entropy	0.743	0.707	0.523
	5	8	60	20	gini	0.742	0.710	0.532
	6	10	80	20	gini	0.741	0.706	0.524

Table 11. Decision Tree Model Evaluation

Random Forest

An ensemble of many decision trees trained on random subsets of the data and features. This model reduces overfitting through averaging. It achieved stable test and OOT results in our tuning runs, with less variance than a single decision tree.

Model		Parameters					Average FDR at 3%		
Random Forest	Iteration	max_depth	min_samples_split	Min_samples_leaf	n_estimators	criterion	Tra	Tst	OOT
	1	5	10	20	10	gini	0.664	0.652	0.539
	2	6	10	20	100	entropy	0.723	0.708	0.576
	3	7	40	15	100	entropy	0.729	0.712	0.575

	4	8	60	20	200	entropy	0.737	0.705	0.574
	5	8	50	15	200	entropy	0.741	0.715	0.575
	6	9	80	20	100	entropy	0.745	0.719	0.577

Table 12. Random Forest Model Evaluation

LightGBM

A gradient boosting framework optimized for speed and performance. It builds trees in a leaf-wise fashion and handles categorical and numerical data well. LightGBM consistently delivered high FDR and strong generalization with limited overfitting.

Model		Parameters				Average FDR at 3%		
LightGBM	Iteration	n_estimators	max_depth	learning_rate	sum_leaves	Tra	Tst	OOT
	1	10	5	0.05	5	0.68	0.665	0.569
	2	20	5	0.05	10	0.708	0.694	0.567
	3	40	6	0.05	19	0.760	0.733	0.567
	4	40	6	0.06	19	0.770	0.733	0.575
	5	50	5	0.06	29	0.776	0.745	0.573
	6	80	4	0.07	22	0.770	0.734	0.575

Table 13. LightGBM Model Evaluation

Neural Network

A multi-layer perceptron model with flexible architecture that can capture complex, non-linear relationships. It performed competitively in our runs, but showed more variability in OOT scores, especially as the hidden layer size increased.

Model		Parameters			Average FDR at 3%		
Neural Network	Iteration	max_iter	learning_rate	hidden_layer_sizes	Tra	Tst	OOT

	1	300	adaptive	(3,3)	0.662	0.668	0.561
	2	200	adaptive	(10,)	0.678	0.671	0.563
	3	300	adaptive	(10,10)	0.705	0.688	0.578
	4	400	adaptive	(50,)	0.723	0.699	0.572
	5	350	adaptive	(20, 20)	0.742	0.713	0.585
	6	500	adaptive	(20, 10, 10)	0.753	0.716	0.582

Table 14. Neural Network Model Evaluation

XGBoost

A powerful and popular boosting algorithm that uses sequential trees and regularization to prevent overfitting. In our case, it achieved the best test FDR among all models, with consistent and tight performance across train, test, and OOT sets.

Model		Parameters				Average FDR at 3%		
XGBoost	Iteration	n_estimators	learning_rate	max_depth	min_child_weight	Tra	Tst	OOT
	1	100	0.05	4	3	0.743	0.728	0.567
	2	200	0.05	4	4	0.778	0.739	0.570
	3	350	0.06	4	6	0.801	0.769	0.577
	4	400	0.04	4	7	0.791	0.754	0.572
	5	200	0.06	5	7	0.798	0.762	0.571
	6	200	0.05	6	8	0.807	0.766	0.576

Table 15. XGBoost Model Evaluation

Model Comparison

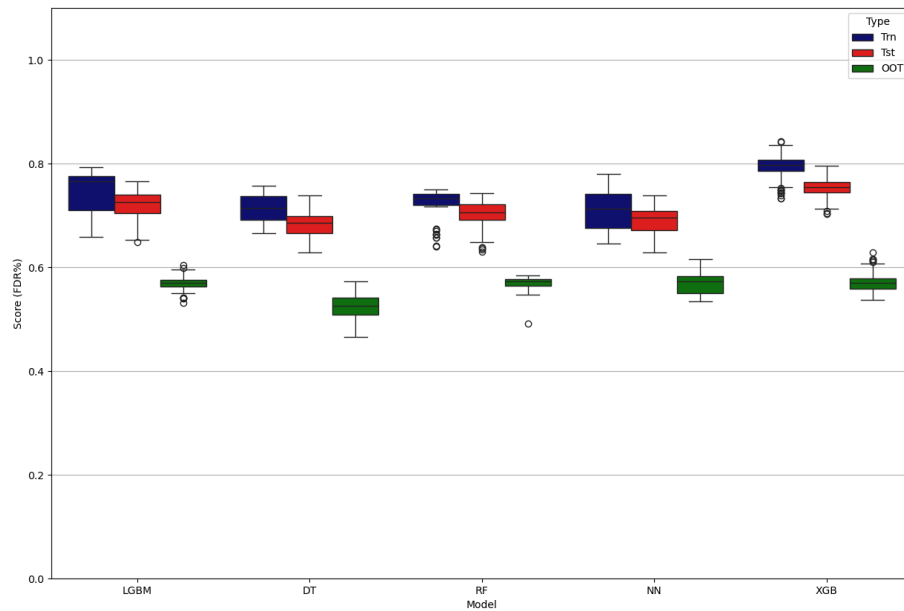


Figure 9. Performance Comparison Across Models

The figure above shows the FDR@3% distributions across six runs for each model on the Train, Test, and Out-of-Time (OOT) datasets. Among all models, **XGBoost (XGB) achieved the highest average FDR on the test set with tight and consistent distributions**, indicating strong generalization and minimal overfitting. LightGBM (LGBM) also performed well, though slightly below XGB overall. Other models like Decision Tree (DT) and Random Forest (RF) exhibited greater variability in OOT performance, while Neural Networks (NN) were competitive but less stable.

OOT Set

This table assesses model performance on out-of-time data, simulating future deployment. Cumulative FDR approaches 90%, and KS peaks above 75. Financial metrics remain consistent, showing that the model maintains good predictive power and financial benefit on unseen data.

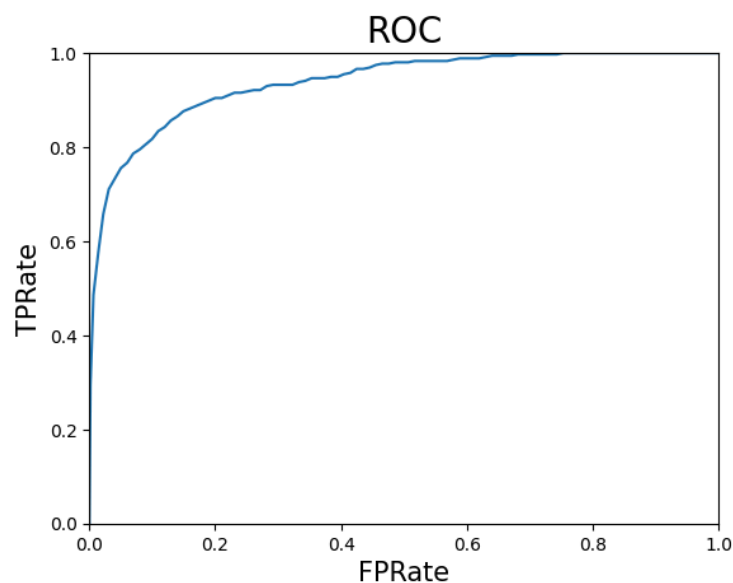
Bin Statistics						Cumulative Statistics						
bin	#recs	#g	#b	%g	%b	tot	cg	cb	%cg	FDR	KS	FPR
1	126	17	109	13.49	86.51	126	17	109	0.14	30.62	30.48	0.16
2	127	59	68	46.46	53.54	253	76	177	0.62	49.72	49.10	0.43
3	126	89	37	70.63	29.37	379	165	214	1.34	60.11	58.77	0.77
4	126	107	19	84.92	15.08	505	272	233	2.21	65.45	63.23	1.17
5	127	112	15	88.19	11.81	632	384	248	3.13	69.66	66.54	1.55
6	126	117	9	92.86	7.14	758	501	257	4.08	72.19	68.11	1.95
7	127	118	9	92.91	7.09	885	619	266	5.04	74.72	69.68	2.33
8	126	114	12	90.48	9.52	1011	733	278	5.97	78.09	72.12	2.64
9	126	121	5	96.03	3.97	1137	854	283	6.95	79.49	72.54	3.02
10	127	114	13	89.76	10.24	1264	968	296	7.88	83.15	75.26	3.27
11	126	122	4	96.83	3.17	1390	1090	300	8.88	84.27	75.39	3.63
12	126	126	0	100.00	0.00	1516	1216	300	9.90	84.27	74.37	4.05
13	127	125	2	98.43	1.57	1643	1341	302	10.92	84.83	73.91	4.44
14	126	120	6	95.24	4.76	1769	1461	308	11.90	86.52	74.62	4.74
15	127	126	1	99.21	0.79	1896	1587	309	12.92	86.80	73.88	5.14
16	126	123	3	97.62	2.38	2022	1710	312	13.92	87.64	73.72	5.48
17	126	122	4	96.83	3.17	2148	1832	316	14.92	88.76	73.85	5.80
18	127	125	2	98.43	1.57	2275	1957	318	15.94	89.33	73.39	6.15
19	126	123	3	97.62	2.38	2401	2080	321	16.94	90.17	73.23	6.48
20	126	125	1	99.21	0.79	2527	2205	322	17.95	90.45	72.49	6.85

Table 18. Out-Of-Time Data Binning Results — Cumulative Performance and Financial Metrics



This plot shows the FDR (Fraud Detection Rate) as a function of the score cutoff percentage. It demonstrates that by reviewing only the top-scoring transactions (e.g., top 3%), the model captures a high percentage of fraudulent cases (around 78–80%), supporting efficient fraud detection with minimal investigation effort.

Figure 10. Fraud Detection Rate Curve



This plot shows the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR) across different classification thresholds. The curve rises quickly and remains close to the top-left corner, indicating strong model discrimination ability and suggesting that the model performs well at identifying frauds with low false positive rates.

Figure 11. ROC Curve

Financial Curves and Recommended Cutoff

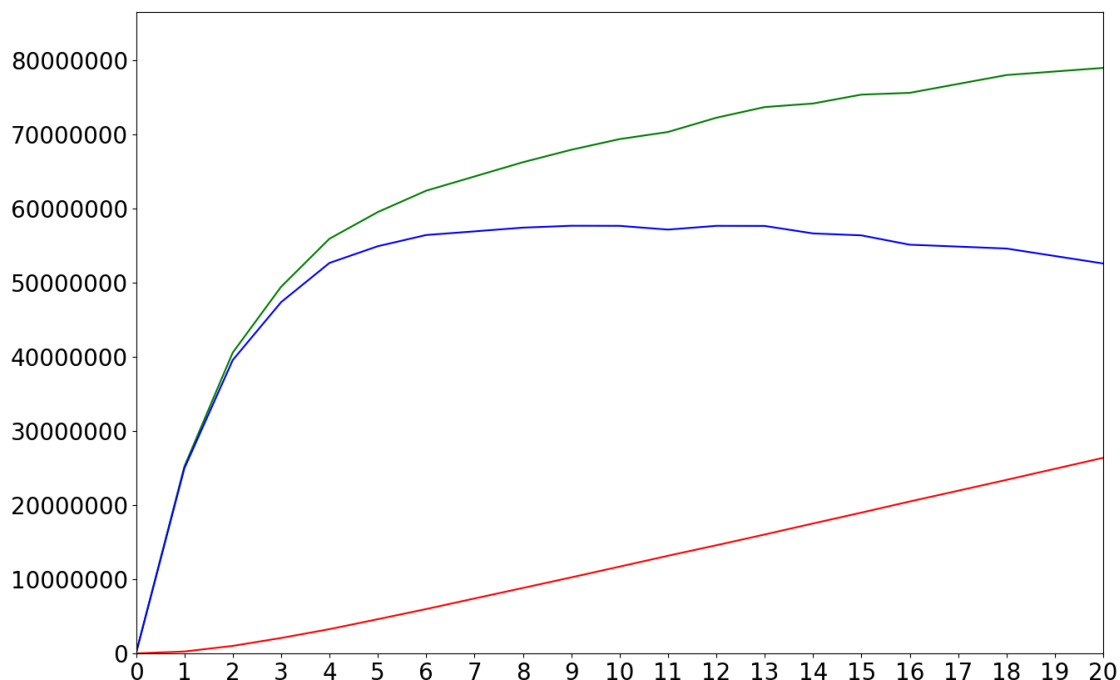


Figure 13. Financial Value Curves

The graph above presents three financial value curves generated from the model's out-of-time predictions. The green line represents the cumulative fraud dollar amount captured, the blue line shows the cumulative number of fraud transactions identified, and the red line indicates the cumulative number of total transactions flagged. All points are sorted in descending order by model score, with each position on the x-axis representing the top ranked transactions..

The green and blue curves increase rapidly in the early stages, showing that the model effectively ranks high-risk transactions at the top. **Both curves begin to flatten after the top 5 to 6 transactions, while the red line continues to grow linearly.** This indicates diminishing returns beyond that point, as additional investigations yield fewer fraud detections.

Based on the graph, **we recommend setting the cutoff around the top 6 to 8 transactions, where the fraud dollar recovery curve (green) begins to flatten.** At this cutoff, the model captures the majority of fraud value with minimal investigation cost, maximizing efficiency and financial impact.

Summary

In this project, I designed and implemented a complete machine learning pipeline to identify high-risk credit card transactions. The dataset included nearly 100,000 transaction records, with only about 2.5% labeled as fraud, presenting a highly imbalanced classification challenge. After thorough data cleaning—including removal of extreme outliers, categorical filtering, and multi-stage imputation—I engineered thousands of features across time, location, and transaction behavior, such as day-since variables, velocity-based counts, and entity interaction patterns.

To reduce redundancy and improve model efficiency, I applied a two-step feature selection process: first using univariate filter scores to keep the top 300 candidates, and then applying wrapper-based forward selection using CatBoost to finalize a diverse and high-performing variable set of 20 fields. I then trained and tuned several models including decision trees, random forests, LightGBM, neural networks, and XGBoost. Each was evaluated using FDR@3% on train, test, and out-of-time (OOT) datasets. Among all candidates, XGBoost consistently produced the best results, with an average FDR@3% of **0.807 (train)**, **0.766 (test)**, and **0.576 (OOT)**.

Finally, I evaluated the financial value of the model using ranked scoring and bin analysis. With only the top 3% of transactions flagged, the model captured approximately 78% of total fraud cases and yielded an estimated **\$59.4 million in** potential fraud savings. The cutoff was selected based on where the fraud dollar recovery curve began to flatten, balancing detection performance with operational cost. Further improvements could include integrating device or network-level data, applying fraud-specific architectures like anomaly detection, or testing post-model rules to refine the flagged transactions.

In future iterations, the model can be integrated with real-time pipelines, or enhanced with additional categorical embeddings or deep learning models for richer transaction context.

Appendix-Full DQR

Data Description

This dataset contains 98,393 credit card transaction records collected from a U.S. government organization, covering a time period from February to October 2010. It includes 10 fields such as transaction date, amount, merchant details, and a binary Fraud label. The data is moderately messy and reflects real-world fraud detection challenges, including missing values, skewed distributions, and ID-like fields that require careful handling during modeling.

Field Summary Tables

Field Name	Field Type	Records Have Value	%Populated	#Zeros	Min	Max	Mean	Standard Deviation	Most Common
Amount	numeric	98,393	100%	0	0.01	3,102,046	424.29092	9,922.44	3.62

Table 1. Numeric Field Summary Table

Field Name	Field Type	Records Have Value	%Populated	#Zeros	Unique Values	Most Common
Date	categorical	98,393	100%	0	365	2/28/10
Merchnum	categorical	94,970	96.52%	0	13,091	930090121224
Merch description	categorical	98,393	100%	0	13,126	GSA-Fss-ADV
Merch state	categorical	97,181	98.77%	0	227	TN
Transtype	categorical	98,393	100%	0	4	P
Recnum	categorical	98,393	100%	0	98,393	1
Fraud	categorical	98,393	100%	95901	2	0
Merch zip	categorical	93,664	95.19%	0	4,567	38118
Cardnum	categorical	98,393	100%	0	1,645	5142148452

Table 2. Categorical Field Summary Table

Field-by-Field Distribution Plots

Recnum

This field is a unique identifier assigned to each individual transaction record. Every value is distinct and appears only once in the dataset. Since it does not have any repeated values or meaningful distribution, it is excluded from plotting and will not be used in modeling.

Cardnum

The Cardnum field represents a unique identifier for the card used in each transaction. This is a high-cardinality categorical field with over 1,600 unique values. The top 20 card numbers account for a significant portion of the total transactions, with the most frequent card appearing over 1,100 times. While the field is not truly unique across rows, the distribution is moderately right-skewed, indicating that a small subset of cards is used much more frequently than others. This field may carry behavioral signals for downstream modeling but should be treated with care to avoid overfitting due to its ID-like nature.

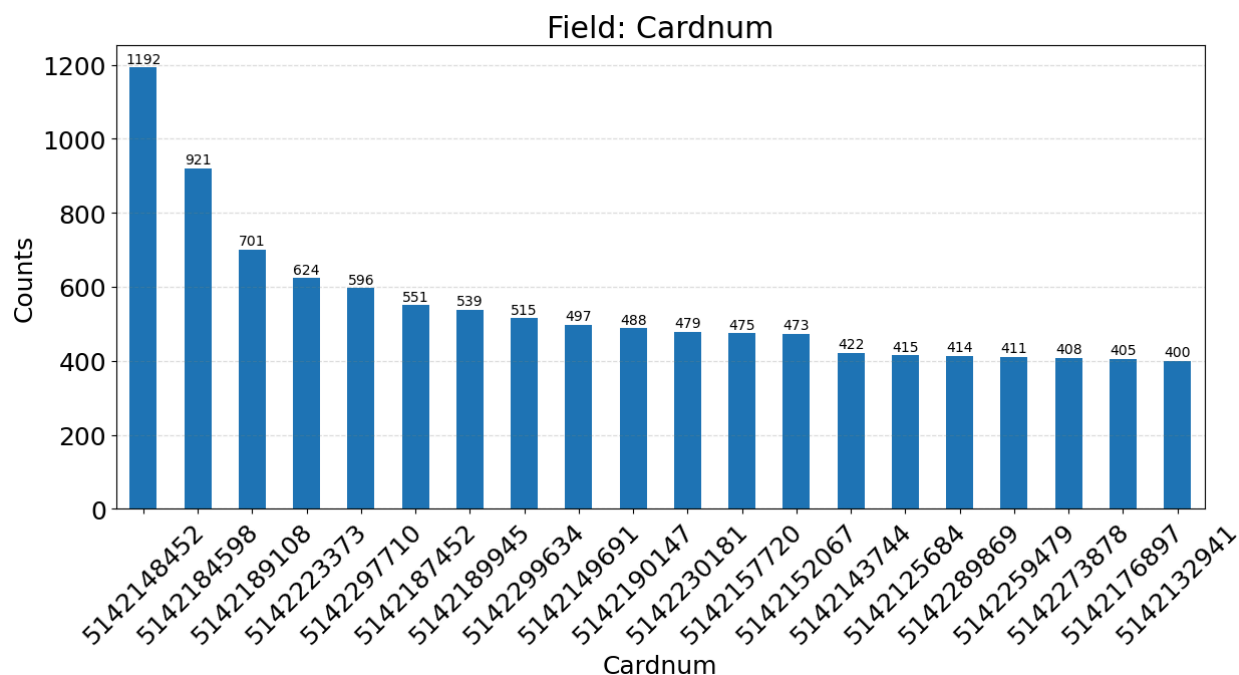


Figure 1. Transtype Distribution

Date

The calendar date of each transaction. This is a datetime field with a wide distribution across the dataset's time span. The top 20 most frequent dates are shown, with relatively consistent transaction volume across dates and a few moderate peaks.

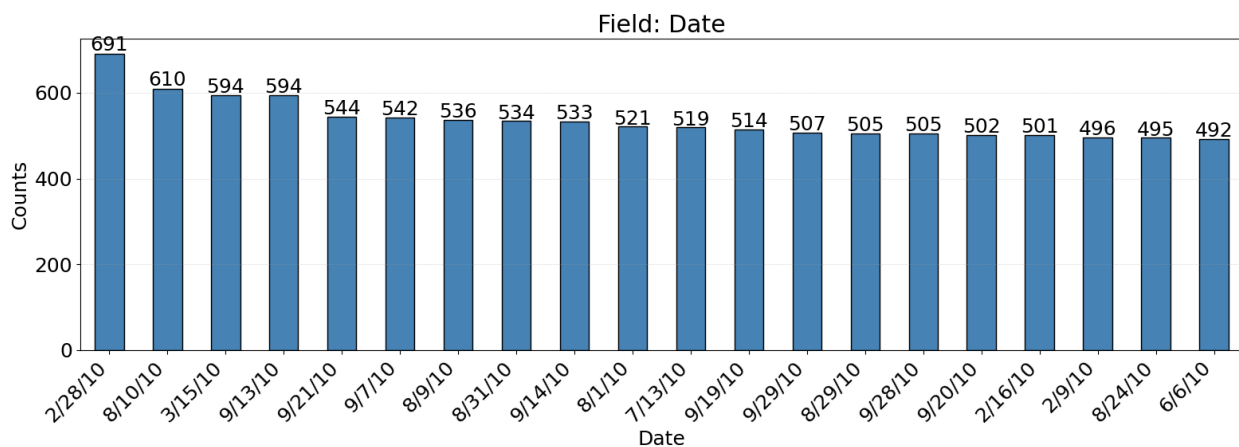


Figure 2. Date Distribution

Merchnum

The Merchnum field represents the unique identifier for each merchant in the dataset. This categorical variable exhibits extreme right-skew, with a small number of merchants accounting for a large proportion of transactions. One merchant alone appears over 9,400 times, while most others appear with far lower frequency. Due to the widespread and class imbalance, a logarithmic y-axis was used in the visualization to more clearly show less frequent merchant IDs. The high cardinality and skew suggest that Merchnum could introduce bias if not properly encoded, and it should be treated carefully if used in modeling.

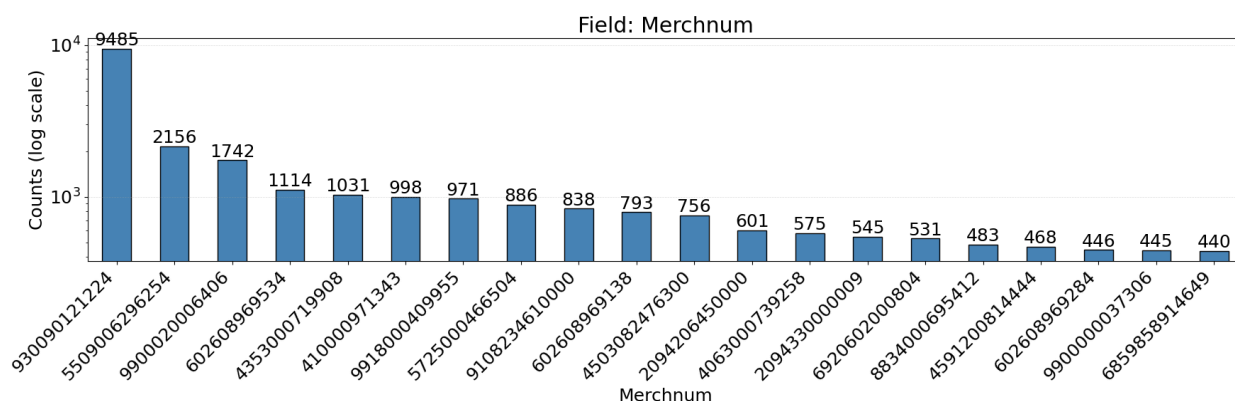


Figure 3. Merchnum Distribution

Merch Description

The name or label of the merchant where the transaction took place. This is a high-cardinality categorical field with many unique values. The top 20 most frequent merchant names are shown, with a gradual decline in frequency after the top few merchants.

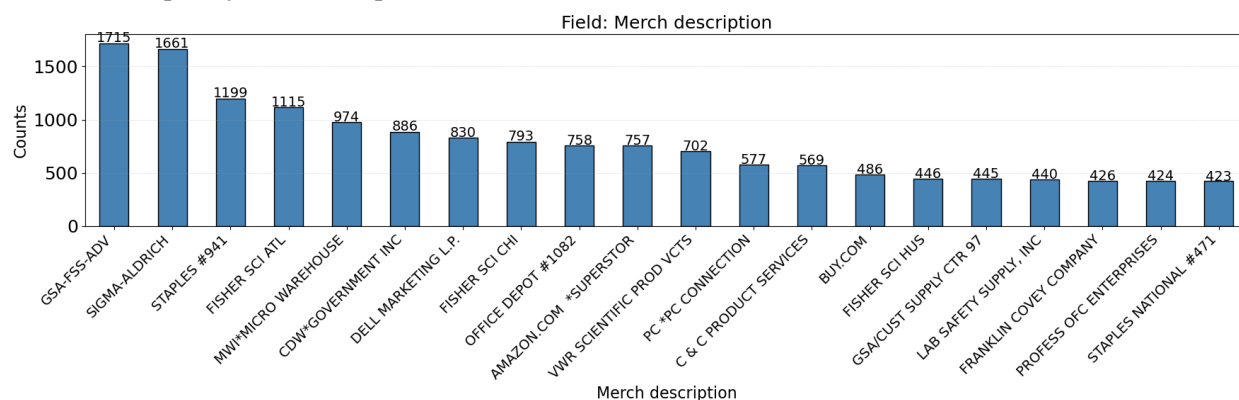


Figure 4. Merch Distribution

Merch State

The Merch state field indicates the U.S. state where the merchant associated with each transaction is located. It is a categorical variable with 227 unique values, though the distribution is highly concentrated. A small number of states account for a large portion of the transactions — notably, Tennessee (TN) dominates the dataset with over 12,000 records, followed by Virginia (VA), California (CA), and Illinois (IL). This skew suggests that the dataset is not geographically balanced, which may reflect internal

procurement patterns of the source organization. The plot shows the top 20 most frequent states, which collectively capture the majority of records.

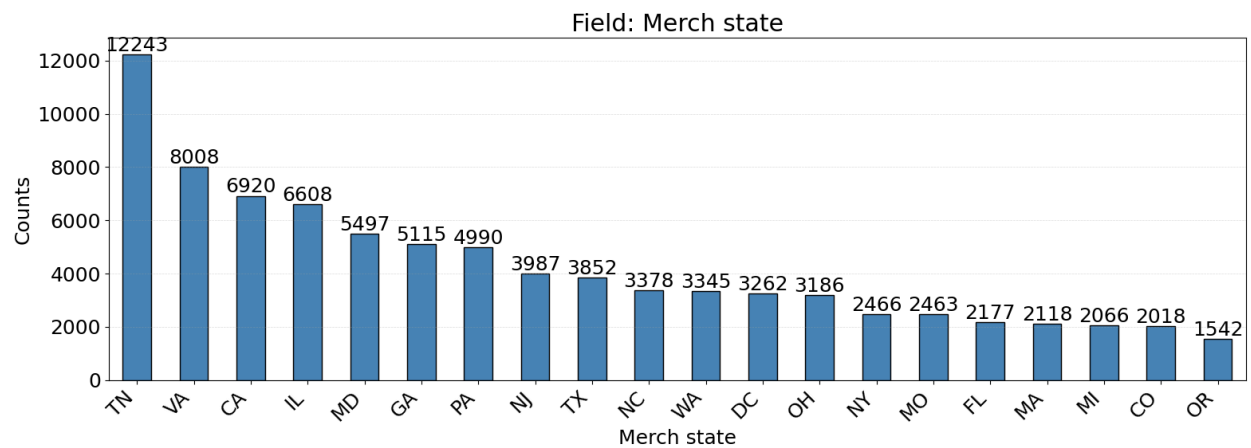


Figure 5. Merch State Distribution

Merch Zip

The Merch zip field represents the ZIP code of the merchant location for each transaction. It is a categorical variable with high cardinality, exhibiting a heavily skewed distribution. One ZIP code (38118) dominates the dataset with over 12,000 records, while the remaining values are far less frequent. The bar chart displays the top 25 ZIP codes by transaction volume, highlighting the geographic concentration of merchant activity. This imbalance suggests that the dataset may be driven by specific regional procurement hubs or vendors, which should be taken into account when analyzing geographic patterns or training models.

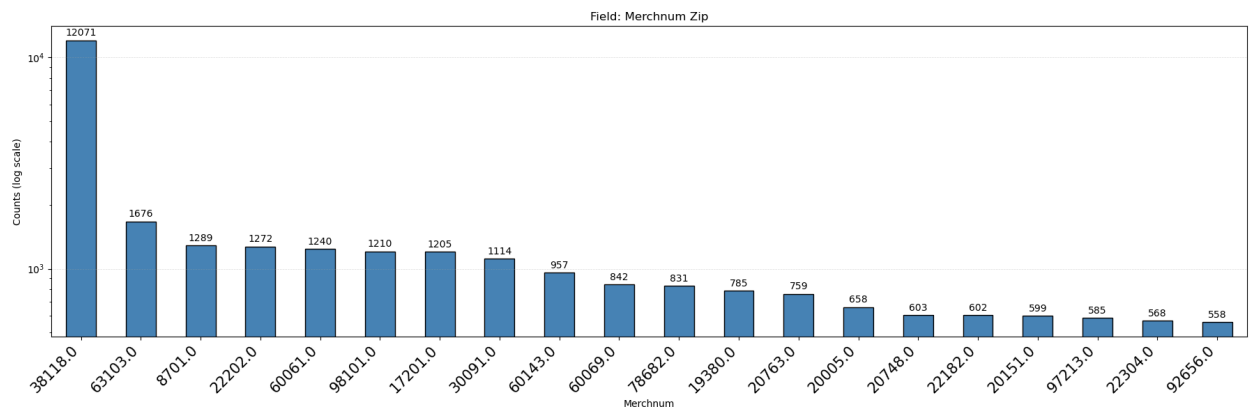


Figure 6. Merchnum Distribution

Transtype

The Transtype field indicates the type of transaction, such as purchase, adjustment, or credit. It is a categorical variable with a strong class imbalance, where one value significantly dominates the distribution. To address this imbalance and enable meaningful comparisons across all categories, a logarithmic y-axis was used in the plot, which helps highlight the presence of low-frequency values that would otherwise be visually obscured.

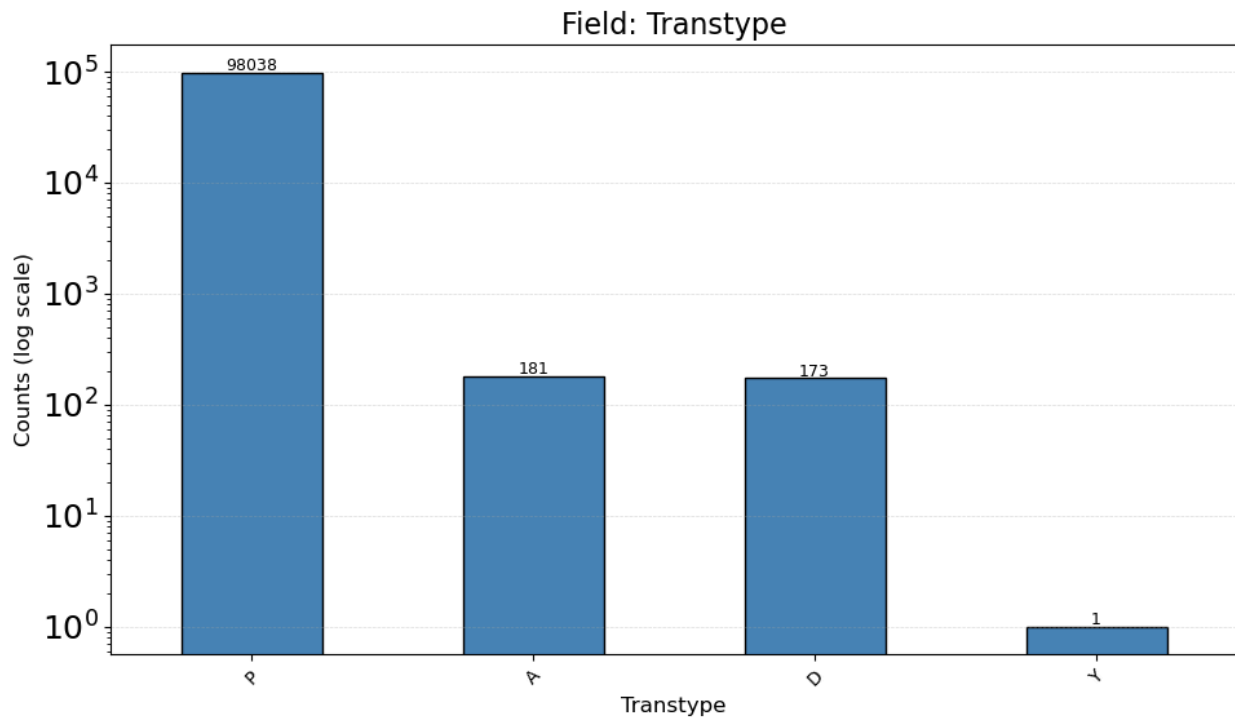


Figure 7. Transtype Distribution

Amount

The Amount field represents the dollar value of each credit card transaction and is a numeric field with a highly right-skewed distribution—most transactions are under \$1,000, while a small number reach much higher values. To make the distribution more interpretable, the data was clipped at the 99.5th percentile to exclude extreme outliers and focus on typical transaction values. A logarithmic scale was applied to the y-axis (frequency) to highlight differences in density across both common and rare transaction amounts, while the x-axis remains on the original dollar scale. This approach effectively reveals the true shape of the transaction distribution without distortion from a few exceptionally large values.

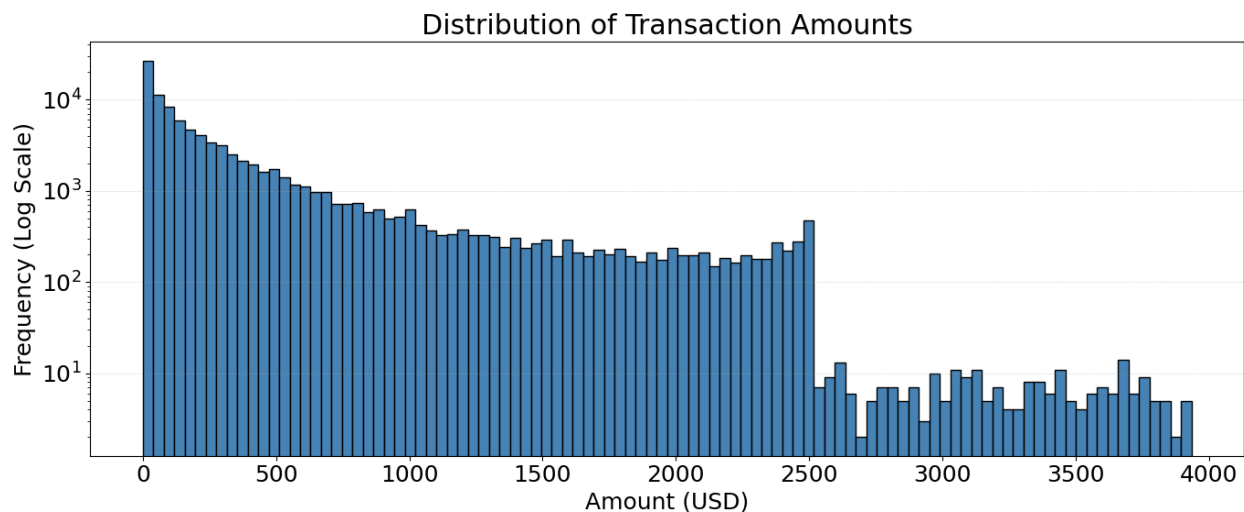


Figure 8. Transaction Amounts Distribution

Fraud

The Fraud field is a binary categorical variable indicating whether a transaction was fraudulent (1) or not (0). Out of the 98,393 total records, only 2,492 (approximately 2.5%) are labeled as fraudulent, while the remaining 95,901 are legitimate. This represents a significant class imbalance, which is typical in fraud detection datasets and presents modeling challenges. The bar chart clearly shows the overwhelming dominance of non-fraudulent transactions. Any modeling efforts will require special consideration of this imbalance (e.g., resampling, using specialized evaluation metrics).



Figure 9.Fraud Distribution